The Case for User-Controlled Cloud Agents

Tyler Ramsarran*

Coding agents, language-model programs that edit files and run commands, execute on user-held compute: machines and allocations their users own or may use. Vendor and research-center documentation shows agents on SSH hosts, customer-run machines, GPU workstations and cluster login nodes, and it marks the limits of these execution paths. In five products a session is bound to one environment. Where vendor products run tools on user-held compute, the vendor runs the agent loop or receives the session's event stream. On the shared research systems reviewed here, each center sets its own agent rules, and agents act with the full reach of the user's account. These limits matter in research computing and in development tasks that span machines and operating systems. This paper argues for a user-controlled cloud agent: like a vendor's cloud session, it keeps running after the user closes the laptop and lets the user steer it from any device. It runs on user-held compute, keeps a multi-machine task under one point of control, follows each site's rules and uses a model endpoint and access the user controls. The paper states five requirements for it.

I. Introduction

A coding agent is a language-model program that edits files and runs commands to complete a software task. Its harness, the software around the model, runs the agent loop. In each pass of the loop, the model plans a step, a tool such as a shell carries it out, and the result returns to the model.

Anthropic's documentation for Claude Code describes where a session can run. A session started from a terminal, an integrated development environment (IDE) or the desktop app with the Local option selected runs on the user's machine. A cloud session runs on infrastructure that Anthropic manages or, when routed there, on an organization's self-hosted environment [1]. The self-hosted case belongs to a wider class, called here user-held compute: compute the user owns or is entitled to use, such as lab workstations, cluster allocations and self-hosted machines. Section II shows vendor and project documentation placing coding agents on user-held compute.

The same documentation also marks the limits of this placement. In vendor products (Section II), five harnesses bind each session or connection to one environment. Runner products, the vendor services that move tool execution to user-held compute, run the agent loop at the vendor or send the vendor the session's event stream. In Anthropic's words, that stream "carries prompts, responses, and tool results" [2]. In research computing (Section III), site policies and data-use rules govern the work, and an agent acts with the full reach of the user's account.

This paper argues for a user-controlled cloud agent. Like a vendor's cloud session, such an agent keeps running after the user closes the laptop, and the user can steer it from any device [1]. It runs its loop on user-held compute, calls a model endpoint the user chooses and holds task-scoped access. It also keeps a multi-machine task under one point of control and follows each site's agent rules. The argument draws only on public sources, retrieved on 9 November 2026. Research computing (Section III) is the lead case and multi-machine development (Section IV) the second.

II. Coding Agents on User-Held Compute

Vendors ship coding agents that execute on user-held compute, and Table 1 records how thirteen harnesses, in three groups, do so. In the runner products of the first group, a worker or runner on the customer's machine executes the session's tools, and the vendor runs the agent loop or receives the session's event stream [2–6]. The second group's harnesses work on a host that the user supplies. Codex, for example, runs chats from the ChatGPT desktop app against the file system and shell of an SSH host [7]. In Cline Desktop, "the agent, its tools, and your code live on the remote machine" [8].

Read across, the execution column records a user-held execution path for ten of the thirteen harnesses. The other three, all in the third group, appear for their model endpoints or for what they send off the machine. The table's columns for machines per session and for what leaves user-held compute show the limits that the rest of this section examines.

*This paper was written with the assistance of artificial intelligence.

Table 1 Execution paths for coding agents on user-held compute, as documented on 9 November 2026.

Group and harnesses	Execution on user-held compute	Machines per session	User-chosen model endpoint	What leaves user-held compute
Runner products: Devin, Cursor, Amp, Claude Code	Devin: commands, edits and repository access on machines the customer operates [3]; Cursor: tool execution on a customer-managed worker [4]; Amp: self-hosted orbs (machines or containers on the customer's platform) run the agent's tools [5]; Claude Code: self-hosted runner, public beta on Team and Enterprise plans [2]	Devin: one worker claims each waiting session [3]; Cursor: machine chosen in the run menu [6]	Claude Code: model calls to the customer's own Bedrock or Google Cloud account [2]	Devin, Amp: agent loop and model inference [3, 5]; Cursor: file contents, terminal output, diffs [4]; Claude Code: prompts, responses, tool results [2]
Harnesses on user hosts: Codex (app and CLI), Cline Desktop, GitHub Copilot, Factory Droid, DeepSeek Harness	Codex: chats against an SSH host's file system and shell [7]; Cline Desktop: agent, tools and code on a remote host [8]; Copilot: a GitHub Actions runner the user configures, or the user's own machine [9]; Factory Droid: the user's Linux, macOS or Windows computer [10]; DeepSeek Harness: files and processes on a remote host over OpenSSH [11]	Codex: one run location per chat; handoff switches the chat to another host [7]; Cline Desktop: one host at a time [8]; Copilot: one environment, fixed at session start [9]; DeepSeek Harness: one remote file system and process world per SSH connection [11]	Codex CLI: local Ollama or LM Studio server, custom providers by base URL [12]; DeepSeek Harness: custom endpoint, including a self-hosted server [13]	Codex (phone to host): prompts, approvals and follow-ups sent to ChatGPT [7]; Factory Droid: connection through Factory's relay [10]; DeepSeek Harness: harness, model transport and sessions stay on the harness host [11]
Local-model and other harnesses: goose, oh-my-pi, Qwen Code, ZCode	goose: the whole loop on one NVIDIA DGX Spark [14]	goose: model runtime and agent on one box [14]	goose: local Ollama model [14]; oh-my-pi: Ollama, LM Studio, llama.cpp, vLLM [15]; Qwen Code: Ollama or vLLM [16]	goose: loopback traffic only, between goose and a local Ollama runtime [14]; ZCode: whole workspaces uploaded to Alibaba Cloud, Repo Wiki feature removed, per Z.ai [17]

The documentation of GitHub Copilot, Cline Desktop, Devin, DeepSeek Harness and Codex binds each session, or in DeepSeek Harness each SSH connection, to one environment. GitHub states for Copilot, "You choose the environment when the session starts, and you can't change it afterward" [9]. Cline Desktop connects to one host at a time, and switching hosts disconnects the current one [8]. In Devin Outposts a worker claims a waiting session and runs its tool calls on its own machine, so that "N workers serve N concurrent sessions" [3]. Each OpenSSH connection in DeepSeek Harness supplies "one remote filesystem/process world" [11].

Codex and Cursor describe how work reaches a chosen machine. A Codex chat can move by handoff, which switches the chat to the destination host and first interrupts any running response [7]. For Cursor, the user selects the machine in the menu that sets where the agent runs [6]. Across Copilot, Cline Desktop, Devin, DeepSeek Harness and Codex, the five harnesses with explicit wording, this paper infers that work on a second machine takes a second session or connection, a switch of host or, in Codex, a handoff.

The vendor's share of each session differs across the runner products. Cognition states that "Devin's agent loop (inference and planning) continues to run in Devin's cloud, while all command execution, file edits, and repository access happen on machines you operate" [3]. Amp and Cursor document the same division of work [4, 5], and Cursor's worker also sends Cursor the file contents, terminal output and diffs that the agent needs [4]. Claude Code's self-hosted runner can send model calls to the customer's own Amazon Bedrock or Google Cloud account. The runner still sends the session's event stream to Anthropic [2].

What leaves user-held compute depends on the harness and the model endpoint as well as on where tools run. In September 2026, The Register reported that developers found Z.ai's ZCode "packaging and git-encrypting entire user workspaces, including complete project histories" and sending them to Alibaba Cloud. Z.ai apologized and said that it had removed the Repo Wiki feature [17].

Model traffic can stay local: the model-endpoint column of Table 1 lists local or self-hosted models for Codex CLI,

goose, oh-my-pi, DeepSeek Harness and Qwen Code. A post on the Agentic AI Foundation blog runs goose against a local model on an NVIDIA DGX Spark. It reports that "the whole loop, reasoning, tool calls, file edits, shell commands, can run without a single packet leaving the machine" [14]. For data under external rules, these choices decide whether prompts and working files stay on user-held compute, a question to which Section III returns for research data.

III. Research Computing

Agents reach research clusters for high-performance computing (HPC), and their commands land on shared login nodes. A login node is the machine where users of a cluster sign in; a batch scheduler such as Slurm then places their jobs on compute nodes. On 17 August 2026, the National Energy Research Scientific Computing Center (NERSC) reported a rise in AI coding agents on Perlmutter login nodes. The agents were overloading file systems and adversely affecting other users, "particularly when find or similar commands are run" [18].

Duke Research Computing explains how the commands get there. Agents such as Claude Code, OpenCode and Codex CLI run on the user's own machine and execute commands over SSH. When they connect directly to the cluster's login host, those commands run on a shared login node [19].

NERSC, Duke, the University of Arizona and UC Davis have each published their own terms for agents, and NERSC's take the form of recommended best practices for running coding agents on its systems [18]. Duke provides an auto-provisioning SSH proxy that directs the agent's commands to a Slurm compute node, and its AI use policy sets out which agentic workflows are acceptable on shared infrastructure [19]. At the University of Arizona, approval is the rule: "AI agents may not independently create, submit, modify, or operate HPC jobs or execute system commands without explicit user approval of the actions being performed" [20]. Every agentic system that runs on or accesses the UC Davis clusters must use the center's cluster skill, a standard agent skill that supplies guidelines for acceptable behavior on its login nodes. The rule extends to a harness on a local machine that reaches the clusters over SSH [21]. A researcher with allocations at Duke, Arizona and UC Davis therefore works under three sets of agent rules.

Researchers and one vendor have built tools that let an agent reach the batch scheduler from the user's side. The ASCEND preprint describes a system, with Claude Code or Codex as the agent, that "runs the agent on the researcher's own laptop, reaching Slurm clusters and a GPU workstation over multiplexed authenticated SSH" [22]. Locally executed tools in the same system check site-specific execution policies [22]. Anthropic's Claude Science connects over SSH to a lab workstation or an HPC login node and, on a host with Slurm, submits jobs to Slurm by default [23].

Agents that reach a cluster over SSH in this way act through the user's account and so hold its full reach. Claude Science states that "Remote jobs run outside the sandbox, as your user on the host, with access to everything your account can read and write there" [23]. The Norwegian HPC provider Sigma2 tells users that "Agents run under your personal account and act on your behalf," including agents on the user's own machine that execute commands on Sigma2 systems over SSH [24]. Cursor's enterprise documentation makes the same point for its local agents: "There is no security boundary between agents and your user account" [25]. Cursor adds that built-in approvals and hooks give most enterprises sufficient control, and it recommends a separate virtual machine, file system permissions or a dedicated development machine for users who need stronger isolation [25]. The University of Arizona addresses credentials directly: "Users must not provide AI agents with credentials in a manner that unnecessarily exposes or transfers control of those credentials" [20].

Data-use rules also limit where prompts may go. A National Institutes of Health notice addresses controlled-access human genomic data. It states that "sharing controlled-access data with public generative AI tools (e.g., third party tools) via prompts or other user interfaces is in violation of the provision on Non-Transferability" [26]. In the runner products of Section II, the vendor runs the agent loop or receives the session's event stream [2, 3]. This paper reads the notice as a reason to keep both the agent loop and the model endpoint under user control for data of this class.

IV. Multi-Machine Development

Multi-machine development is the second case, and Microsoft and Epic Games document iOS builds that span a Windows machine and a Mac [27, 28]. Microsoft's documentation states that "Building native iOS applications using .NET Multi-platform App UI (.NET MAUI) requires access to Apple's build tools, which only run on a Mac" [27]. Because of this, Visual Studio 2022 connects over SSH to a Mac on the network, so that the developer can work on Windows [27]. Epic Games documents the same arrangement for Unreal Engine, which can connect a Windows PC over SSH to up to two Macs to create iOS builds [28]. An IDE or an engine runs these build steps. Cursor lists the same hardware among its reasons to bring a self-hosted machine: "You have custom hardware, such as GPU machines or

Macs for iOS development" [4].

The Diamond Agent preprint states a cross-system need for research computing. It describes a centralized Diamond Agent instance that can operate several supercomputers [29]. One of its design requirements reads, "Task state, results, and artifacts produced on one system must remain accessible to later workflow stages on another" [29].

One developer's account describes agents running on several machines at once, and Codex and Cursor let one user register several machines. The developer writes, "I often have three or four coding agents running across different machines," and describes a session manager built to keep track of them and to use them from a phone [30]. Codex instructs the user to pair each phone or desktop device with every host it should control, and the user switches between connected hosts and chats [7]. Cursor's setup guide advises a friendly name for each machine when a user has several for the same repository [6]. This paper infers from Section II that, in the harnesses it names, a task that needs both a Windows machine and a Mac becomes two sessions whose shared context the user carries between them. In Codex, a handoff moves the chat and its Git state from one host to the other [7].

V. Requirements

The evidence of Sections II to IV indicates five requirements for a user-controlled cloud agent. Requirement 1 sets where the agent runs; Requirements 2 to 5 address the reach of one session, the rules of each site, who holds the loop and the model endpoint, and the reach of the user's account.

1) Placement: the agent must run where the user holds compute, such as a lab workstation, a cluster allocation, a self-hosted machine, a Mac or a GPU machine (Sections II to IV).
2) Multi-machine tasks: the agent must keep a task that needs more than one machine or operating system under one point of control, which holds the task's context across those machines (Sections II and IV).
3) Site rules: the agent must reach shared compute through the site's batch scheduler, follow the site's agent policy and stop for human approval where the site requires it (Section III).
4) User-side control path: the agent must keep its loop, working files and the session's event stream on user-held compute and call a model endpoint the user chooses, including a local or self-hosted model (Sections II and III).
5) Scoped access: the agent must act with credentials and file access scoped to its task, below the full reach of the user's account (Section III).

ASCEND and Claude Science meet parts of Requirements 1 and 3 by reaching the batch scheduler from the user's side, and the harnesses that Table 1 lists with a local or self-hosted model meet the model-endpoint half of Requirement 4. Diamond Agent states the cross-system need behind Requirement 2. This paper's position is that a single agent should meet all five requirements together.

VI. Conclusion

The documentation reviewed here shows vendors and research centers placing coding agents on user-held compute. The limits lie in who holds the agent loop, how far one session reaches and what each site allows an agent to do with the user's account. The five requirements of Section V state what an agent must do to work within these limits in research computing and multi-machine development. The author's Hermes Agent plugin, Own Compute, is an early implementation written toward these requirements.

References

[1] Anthropic, "Use Claude Code in the Cloud," Claude Code Docs. https://code.claude.com/docs/en/claude-code-on-the-web [retrieved 9 November 2026].

[2] Anthropic, "Customize Sessions in Self-Hosted Environments," Claude Code Docs. https://code.claude.com/docs/en/self-hosted-environments-configuration [retrieved 9 November 2026].

[3] Cognition, "Devin Outposts: Self-Hosted Infrastructure," Devin Docs. https://docs.devin.ai/cloud/outposts/overview [retrieved 9 November 2026].

[4] Cursor, "Self-Hosted Machines," Cursor Docs (Cloud Agents). https://cursor.com/docs/cloud-agent/self-hosted [retrieved 9 November 2026].

[5] Amp, "Self-Hosted Orbs," Amp Docs. https://ampcode.com/docs/orbs/self-hosted [retrieved 9 November 2026].

[6] Cursor, "My Machines," Cursor Docs (Cloud Agents). https://cursor.com/docs/cloud-agent/self-hosted/my-machines [retrieved 9 November 2026].

[7] OpenAI, “Remote Connections,” ChatGPT Learn (Codex). https://learn.chatgpt.com/docs/remote-connections [retrieved 9 November 2026].

[8] Cline, “Remote Development over SSH,” Cline Docs (Cline Desktop). https://docs.cline.bot/usage/cline-desktop-ssh [retrieved 9 November 2026].

[9] GitHub, “About Cloud Sandboxes for GitHub Copilot,” GitHub Docs. https://docs.github.com/en/copilot/concepts/security-governance-and-network-settings/about-cloud-sandboxes#choosing-where-copilot-runs [retrieved 9 November 2026].

[10] Factory, “Remote Control,” Factory Docs. https://docs.factory.com/remote-control [retrieved 9 November 2026].

[11] DeepSeek, “SSH,” DeepSeek Harness Docs, GitHub (deepseek-ai/deepseek-harness). https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/subsystems/ssh.md [retrieved 9 November 2026].

[12] OpenAI, “Advanced Configuration,” ChatGPT Learn (Codex). https://learn.chatgpt.com/docs/config-file/config-advanced [retrieved 9 November 2026].

[13] DeepSeek, “Configure Models,” DeepSeek Harness User Guide, GitHub (deepseek-ai/deepseek-harness). https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/user/guide/providers.md [retrieved 9 November 2026].

[14] Pathak, S., “Running goose Fully Offline on a DGX Spark,” Agentic AI Foundation Blog, 17 July 2026. https://aaif.io/blog/running-goose-fully-offline-on-a-dgx-spark [retrieved 9 November 2026].

[15] Stencil Labs, “README,” GitHub (can1357/oh-my-pi). https://github.com/can1357/oh-my-pi/blob/main/README.md [retrieved 9 November 2026].

[16] QwenLM, “README,” GitHub (QwenLM/qwen-code). https://github.com/QwenLM/qwen-code/blob/main/README.md [retrieved 9 November 2026].

[17] Jones, C., “Z.ai Says Sorry for Slurping Up Your Code, Open Sources ZCode,” The Register, 22 September 2026. https://www.theregister.com/security/2026/09/22/zai-says-sorry-for-slurping-up-your-code-open-sources-zcode/5298300 [retrieved 9 November 2026].

[18] NERSC, “Coding Agent Impacts Prompt Best Practices Recommendations,” National Energy Research Scientific Computing Center, 17 August 2026. https://www.nersc.gov/users/user-news/coding-agent-impacts-prompt-best-practices [retrieved 9 November 2026].

[19] Duke Research Computing, “Access via AI Agents: Duke Compute Cluster,” Duke Research Computing Support Docs. https://oit-rc.pages.oit.duke.edu/rcsupportdocs/dcc/ai-access/ [retrieved 9 November 2026].

[20] University of Arizona Research Computing, “Agentic AI,” UArizona HPC Documentation (Policies). https://hpcdocs.hpc.arizona.edu/policies/agentic_ai [retrieved 9 November 2026].

[21] HPC@UCD, “Agentic AI,” HPC@UCD Documentation (Policies). https://docs.hpc.ucdavis.edu/policies/agentic-ai/ [retrieved 9 November 2026].

[22] Liu, J. P., Herath, U., and Petersen, A., “ASCEND: Personal AI Agents for Autonomous Scientific Computing Across HPC Clusters and GPU Workstations,” arXiv:2609.32868 [cs.DC], 26 September 2026. https://arxiv.org/abs/2609.32868 [retrieved 9 November 2026].

[23] Anthropic, “Remote Compute Clusters,” Claude.ai Documentation (Claude Science). https://claude.com/docs/claude-science/remote-compute-clusters [retrieved 9 November 2026].

[24] Sigma2, “Policy for Agentic AI Tools on HPC Systems,” 29 September 2026. https://www.sigma2.no/policy-agentic-ai-tools-hpc-systems [retrieved 9 November 2026].

[25] Cursor, “LLM Safety and Controls,” Cursor Docs (Enterprise). https://cursor.com/docs/enterprise/llm-safety-and-controls [retrieved 9 November 2026].

[26] National Institutes of Health, “NOT-OD-25-081: Protecting Human Genomic Data When Developing Generative Artificial Intelligence Tools and Applications,” NIH Grants and Funding, 28 March 2025. https://grants.nih.gov/grants/guide/notice-files/NOT-OD-25-081.html [retrieved 9 November 2026].

[27] Microsoft, “Pair to Mac for iOS Development,” Microsoft Learn (.NET MAUI), 31 July 2025. https://learn.microsoft.com/en-us/dotnet/maui/ios/pair-to-mac?view=net-maui-10.0 [retrieved 9 November 2026].

[28] Epic Games, “Creating Remote Builds of Unreal Engine Projects for iOS,” Unreal Engine 5.9 Documentation. https://dev.epicgames.com/documentation/en-us/unreal-engine/creating-remote-builds-of-unreal-engine-projects-for-ios [retrieved 9 November 2026].

[29] Xie, H., Chen, J., Zheng, M., Zhu, Y., Mathew, M., Burnette, M., Babuji, Y., Kindratenko, V., Venkataraman, S., Chard, K., Foster, I., and Zhang, Z., “Diamond Agent: Agentic Control of Federated HPC Resources as a Service,” arXiv:2609.06181 [cs.DC], 5 September 2026. https://arxiv.org/abs/2609.06181 [retrieved 9 November 2026].

[30] Kuznetsov, S., “ccmux: Managing AI Coding Agents Across Devices,” blog.skz.dev, 5 June 2026. https://blog.skz.dev/ccmux-tmux-manager-for-ai-coding-agents [retrieved 9 November 2026].